CURSO HTML 5 / CSS 3

Este obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-CompartirIgual 3.0 Unported.

Enpresa Digitala

ÍNDICE:

HTML 5 ..... **3**

PARTE 1: NOVEDADES *3*
1. DECLARACIÓN Y CODIFICACIÓN 4
2. ELEMENTOS ESTRUCTURALES 5
3. OTROS ELEMENTOS 12
4. FORMULARIOS 14
4. AUDIO Y VIDEO 20
5. CANVAS 23
7. API's 26

PARTE 2: SOPORTE *28*
1. SOPORTE EN NAVEGADORES 29
2. MÁS RECURSOS 30

CSS 3 ..... **31**

PARTE 1: INTRODUCCIÓN CSS 3 ..... *31*
1. INTRODUCCIÓN: 32

PARTE 2: NOVEDADES *34*
1. BORDES REDONDEADOS 35
2. IMÁGENES EN BORDES 37
3. AJUSTES DE LOS BORDES 39
4. MÚLTIPLES IMÁGENES EN EL FONDO 40
5. SOMBRAS EN BLOQUES 41
6. SOMBRAS EN TEXTOS 42
7. CONTORNO DE TEXTO 43
8. CONTORNO DE BLOQUES 44
9. TIPOGRAFÍAS PERSONALIZADAS 45
10. RGBA 46
11. HSLA 48
12. COLUMNAS MÚLTIPLES 49
13. TAMAÑO DE BLOQUE 50
14. REDIMENSIONAMIENTO DE BLOQUE 51
15. GRADIENTES 52
16. REFLEJOS y MÁSCARAS 54
17. FILTROS 57
18. TRANSFORMACIONES 2D 58
19. TRANSFORMACIONES 3D 60
20. TRANSICIONES 61
21. KEYFRAMES 63
22. MEDIA QUERY 65

PARTE 3: SELECTORES *66*
1. LISTA DE SELECTORES 67

Parte 4: SOPORTE Y RECURSOS *69*
1. SOPORTE 70
2. RECURSOS ÚTILES 71
3. BONUS 73

Enpresa Digitala

HTML 5

PARTE 1: NOVEDADES

1. DECLARACIÓN Y CODIFICACIÓN

El `doctype` se reduce a la mínima expresión:

```
<!DOCTYPE html>
```

Lo mismo con la codificación de caracteres:

```
<meta charset="utf-8">
```

Por lo tanto, si en nuestra plantilla base antes usábamos:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-type" content="text/html;
charset=utf-8" />
    <title>XHTML 1.0</title>
  </head>
  <body>
    <h1>Hola</h1>
  </body>
</html>
```

Ahora lo vamos a reducir a:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>HTML5</title>
  </head>
  <body>
    <h1>Hola</h1>
  </body>
</html>
```

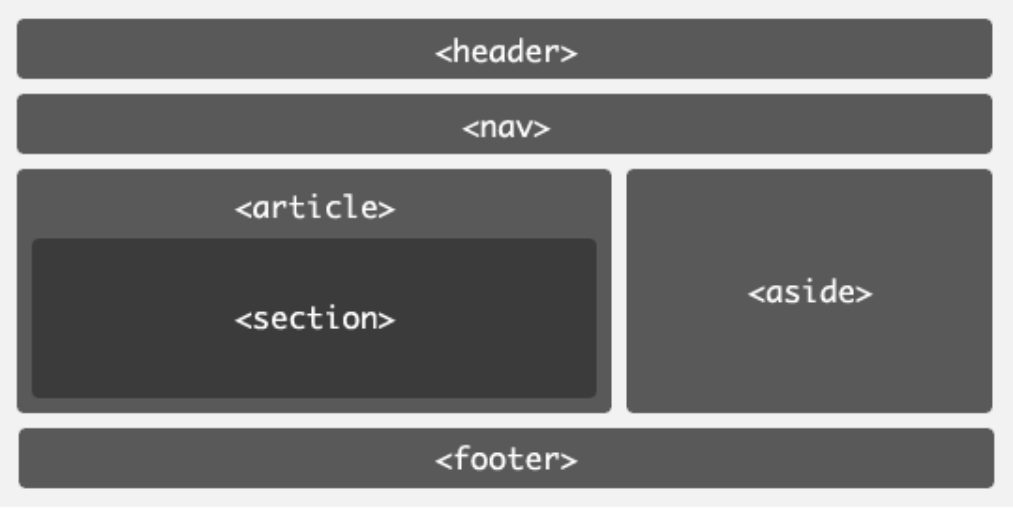
2. ELEMENTOS ESTRUCTURALES

Hasta ahora, utilizábamos el elemento `<div>` para agrupar diversos bloques información y aplicar propiedades CSS. Sin embargo, estos bloques no tenían ningún significado semántico.

Con los nuevos elementos estructurales de HTML 5, vamos a poder sustituir la mayor parte de los elementos `<div>` y dotar a nuestro documento de una estructura semántica adecuada.

Antes ("divitis", en muchos casos, aguda):

Ahora (utilizaremos los elementos estructurales , cuando sea oportuno, para dar sentido y significado a los contenidos):



ARTICLE:

El elemento `<article>` representa una composición autónoma en un documento, página, aplicación o sitio web, con intención de poder ser reutilizado (por ejemplo, en un RSS).

Puede utilizarse en un artículo de un foro, revista, artículo de periódico, entrada (post) de un blog, comentario escrito por un usuario, widget interactivo o gadget e incluso en cualquier otro elemento independiente de contenido.

Cuando existen elementos `<article>` anidados, los elementos `<article>` interiores estarían relacionados con los `<article>` exteriores (p.e un `<article>` que contenga un artículo de un blog, contendrá también `<article>` para los comentarios de un usuario).

Antes:

```
<div class="post">
    <h2>Apúntate al curso de CSS 3 y HTML 5</h2>
    […contenido del post…]
</div>
```

Ahora:

```
<article class="post">
    <h2>Apúntate al curso de CSS 3 y HTML 5</h2>
    […contenido del post…]
</article>
```

SECTION:

El elemento `<section>` representa una sección genérica de un documento o aplicación. Una sección, en este contexto, es una agrupación temática de contenido, habitualmente con un encabezado.

Se recomienda usar `<article>` en vez de `<section>` cuando tenga sentido sindicar los contenidos del elemento.

El elemento `<section>` no es un elemento contenedor genérico. Si sólo se necesita el elemento para aplicar estilos, entonces se deberá utilizar el elemento `<div>`.

Antes:

```
<div id="bloque-noticias">
    [listado de las 5 noticias más recientes]
</div>

<div id="bloque-eventos">
    [listado de los 5 eventos más recientes]
</div>

<div id="bloque-entrevistas">
    [listado de las 5 entrevistas más recientes]
</div>
```

Ahora:

```
<section id="bloque-noticias">
    [listado de las 5 noticias más recientes]
</section>

<section id="bloque-eventos">
    [listado de los 5 eventos más recientes]
</section>

<section id="bloque-entrevistas">
    [listado de las 5 entrevistas más recientes]
</section>
```

HEADER:

El elemento `<header>` representa un grupo introductorio de información o también puede usarse para agrupar ayudas de navegación.

En principio está pensado para que se incluya el encabezado de sección (`<h1>`-`<h6>` o `<hgroup>`), pero no es obligatorio.

También puede usarse para agrupar la tabla de contenidos de una sección, un formulario de búsqueda o logotipos relevantes.

El elemento `<header>` no crea una nueva sección

Antes:

```
<div id="cabecera">
    <h1>Nombre Empresa</h1>
    [menú idiomas, auxiliar...]
</div>
```

Ahora:

```
<header>
    <h1> Nombre Empresa </h1>
    [menú idiomas, auxiliar...]
</header>
```

FOOTER:

El elemento `<footer>` representa el pie de la sección de contenido predecesora más cercana, o la sección del contenido raiz.

Normalmente, el elemento `<footer>` contiene información acerca de su sección, como el autor, enlaces a documentos relacionados, licencias, etc.

También es posible incluir la información que habitualmente se incluye en el elemento `<address>` (información de contacto del autor o editor, por ejemplo).

El elemento `<footer>` no tiene por que aparecer en el final de la sección.

Cuando el elemento `<footer>` contiene secciones enteras, éstas representan apéndices, índices, acuerdos de licencias y otro contenido similar.

El elemento `<footer>` no crea una nueva sección.

Antes:

```
<div id="pie">
    <p> Copyright.. ./ imagen... / texto....</p>
</div>
```

Ahora:

```
<footer>
    <p> Copyright... /  imagen... /  texto....</p>
</footer>
```

NAV:

El elemento `<nav>` representa una sección de la página que contiene enlaces de navegación a zonas de la misma página o a otras páginas.

No todos los grupos de enlaces de una página tienen que implementarse con el elemento `<nav>`. Sólo aquellas secciones que contengan los bloques principales de navegación.

En el caso de los típicos enlaces del "menú auxiliar" (portada, aviso legal, etc…), con el elemento `<footer>` sería suficiente (aunque podemos utilizar además, `<nav>`).

Algunos agentes de usuario (como los lectores de pantalla, por ejemplo), omitirán la lectura de los elementos `<nav>` en una primera lectura (como si hubiéramos implementado un salto[1] de contenido).

Antes:

```
<div id="menu-ppal">
    <ul>
      <li><a href="#">quiénes somos</a></li>
      <li><a href="#">servicios</a></li>
      <li><a href="#">noticias</a></li>
    </ul>
</div>
```

Ahora:

```
<nav id="menu-ppal">
  <ul>
    <li><a href="#">quiénes somos</a></li>
    <li><a href="#">servicios</a></li>
    <li><a href="#">noticias</a></li>
  </ul>
</nav>
```

[1] http://www.webaim.org/techniques/skipnav/

ASIDE:

El elemento `<aside>` representa una sección de la página cuyo contenido está relacionado tangencialmente con el contenido que tiene a su alrededor, pero se considera contenido independiente, adicional.

Se puede usar para implementar barras laterales, publicidad, elementos con efectos tipográficos (como las citas)

No es adecuado utilizarlo para estructurar contenido explicativo, porque forma parte del contenido principal.

Antes:

```
<div id="publicidad">
    [... diferentes bloques de publicidad...]
</div>
```

Ahora:

```
<aside id="publicidad">
    [... diferentes bloques de publicidad...]
</aside>
```

FIGURE / FIGCAPTION:

El elemento <figure> permite asociar contenido embebido (diagramas, ilustraciones, fotos, video, audio, citas…) con un texto.

El elemento <figcaption> se utilizará para implementar el texto asociado al contenido embebido.

Antes:

```
<div id="bloque-foto">
    <img...>
    Pie de foto
</div>
```

Ahora:

```
<figure>
    <img...>
    <figcaption>
       Pie de foto
    </figcaption>
</figure>
```

TIME:

El elemento `<time>` representa una fecha/hora[2] en base al calendario Gregoriano.

(El elemento desapareció[3] de la especificación en octubre de 2011[4] y volvió al mes siguiente[5])

En la especificación HTML 5 se contemplan diferentes maneras[6] de implementar las fechas/horas.

Antes:

```
<div class="post">
    <h2>Apúntate al curso de CSS 3 y HTML 5</h2>
    <p class="fecha">26 de abril de 2012</p>
    [...contenido del post...]
</div>
```

Ahora:

```
<article class="post">
    <h2>Apúntate al curso de CSS 3 y HTML 5</h2>
    <p>
      <time datetime="2012-04-26" pubdate>
            26 de abril de 2010
      </time>
    </p>
    [...contenido del post...]
</article>
```

PICTURE:

El elemento `<picture>`[7] es de reciente creación y tiene como objetivo dar una solución al Responsive Design de las imágenes. Aún está en desarrollo y el soporte es prácticamente nulo:

```
<picture width="300" height="200">
   <source media="(min-width: 900px)" src="grande.jpg">
   <source media="(min-width: 550px)" src="mediana.jpg">
   <source src="peque.jpg">
   <img src="peque.jpg" alt="">
   <p>Texto Accessible Alternativo</p>
</picture>
```

Y mediante el atributo `srcset`[8] (también de nueva creación), es posible discriminar las imágenes en base a la densidad de píxel (retina):

---

2 http://www.whatwg.org/specs/web-apps/current-work/multipage/text-level-semantics.html#the-time-element
3 http://www.iandevlin.com/blog/2011/10/html5/on-the-disappearance-of-html5
4 http://www.brucelawson.co.uk/2011/goodbye-html5-time-hello-data/
5 http://www.brucelawson.co.uk/2011/the-return-of-time/
6 http://www.whatwg.org/specs/web-apps/current-work/multipage/common-microsyntaxes.html#valid-global-date-and-time-string
7 http://www.w3.org/TR/html-picture-element/

```
<picture width="300" height="200">
   <source media="(min-width: 900px)" srcset=" grande-1.jpg 1x,
grande-2.jpg 2x">
   <source media="(min-width: 550px)" srcset=" mediana-1.jpg 1x,
mediana-2.jpg 2x">
   <source srcset=" peque-1.jpg 1x, peque-2.jpg 2x">
   <img src=" peque-1.jpg" alt="">
   <p>Texto Accessible Alternativo</p>
</picture>
```

EJEMPLO GLOBAL:

Página en HTML 4[9] vs. Página en HTML 5[10]

8 http://www.w3.org/html/wg/drafts/srcset/w3c-srcset/

9 http://diveintohtml5.info/examples/blog-original.html

10 http://diveintohtml5.info/examples/blog-html5.html

3. OTROS ELEMENTOS

Algunos elementos son nuevos y otros se han reutilizado, dejando atrás su función como elemento de presentación para convertirse en elementos semánticos[11] [12].

b

El elemento `<b>` representa un fragmento de texto que se resalta por ser importante en relación al contenido: palabras clave de un documento, nombres de productos en una revisión del mismo, o cualquier otro fragmento de texto que se represente en negrita en modo escrito.

```
<h2>Caracoles Lentos</h2>
<p>Los <b class="nom-product">Caracoles Lentos</b> son un nuevo
producto de la empresa alimenticia....</p>
```

i

El elemento `<i>` representa un fragmento de texto en voz alternativa o contenido que se presenta con itálica en modo escrito: palabras en otro idioma (utilizando el atributo `lang`), términos técnicos y taxonómicos, notación musical, pensamientos, cambios en el estado de ánimo, referencia a contenido escrito a mano...

Es recomendable utilizar clases para indicar el significado que se le quiere dar al elemento `<i>` en cada caso (de este modo se demuestra que no se está utilizando como elemento de diseño):

```
<p>Ayer probé un plato de <i lang="ca">escargots</i>, caracoles
de la familia de los <i class="taxonomy">Helix aspersa</i>.</p>
```

small

El elemento `<small>` se utilizará para el contenido llamado comúnmente de "letra pequeña" (avisos legales y similares) y para comentarios adicionales a un texto (no confundirlo con el elemento `<aside>`).

En definitiva, para aquellos contenidos que en modo escrito se representan con letra más pequeña.

```
<small>
     <a rel="license" href="http://creativecommons.org/licenses/by-sa/3.0/">
        Creative Commons Attribution Share-alike license
     </a>
</small>
```

```
<p> El diseño está basado en la plantilla Keko de Mkels
     <small>(http://www.mkels.com/demo/)</small>
</p>
```

11 http://html5doctor.com/i-b-em-strong-element/

12 http://html5doctor.com/small-hr-element/

hr

El elemento <hr> representa una separación temática entre contenidos.

Otros elementos

Debido a que todavía no hay un borrador definitivo sobre la especificación HTML 5, multitud de nuevos elementos están continuamente apareciendo y desapareciendo de la especificación.

Siempre podemos consultar el Glosario[13] de HTML 5.

`<mark>`: representa un contenido remarcado[14] que es relevante para el usuario.

`<details>`: marca un texto como información adicional.

`<command>`: representa un comando que el usuario puede ejecutar.

`<ruby>`: permite la inserción de anotaciones para lenguajes asiáticos.

13 http://html5doctor.com/glossary/

14 http://html5doctor.com/draw-attention-with-mark/

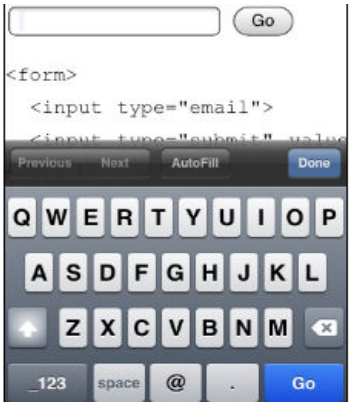
4. FORMULARIOS

HTML 5 aumenta los tipos de campos de formulario[15] y ofrece nuevas funcionalidades[16].

input:

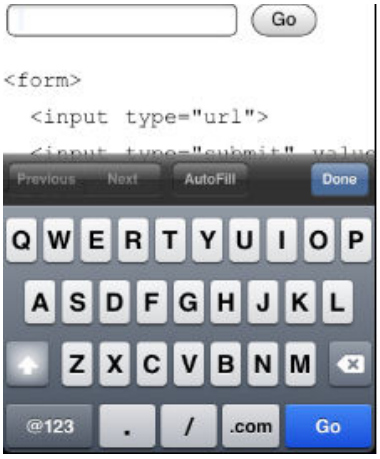
email:

Dirección de e-mail.



url:

Dirección URL.



tel:

Número telefónico

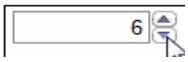
15 http://diveintohtml5.info/forms.html

16 http://miketaylr.com/pres/html5/forms2.html

number:

Aumenta o añade un número, mediante botones:

```
<input type="number" min="0" max="6" step="1" value="6">
```



range:

Selecciona un valor dentro del rango especificado:

```
<input type="range" min="0" max="10" step="1" value="6">
```

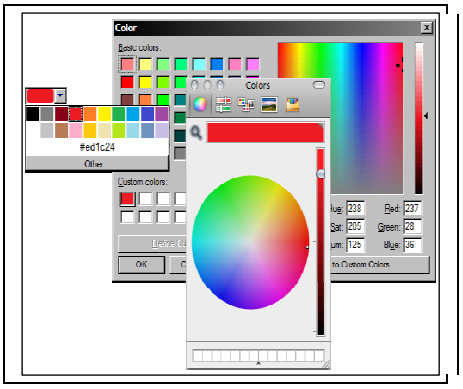
datetime /time / date / month / week:

Selección de Fecha, hora, día, mes o semana concreto en un calendario:

```
<input type="datetime">
```

color:

Provee mecanismos para que el usuario inserte un color RGB



search:

Campo de buscador.

datalist

Se carga una lista de opciones asociadas a un campo input y permite un autocompletado del texto que escribe el usuario, si éste se encuentra en la lista de opciones que ofrecemos[17]:

```
<input list="deportes">
<datalist id=" deportes">
    <option value="Fútbol">
    <option value="Baloncesto">
    <option value="Tenis">
    <option value="Motociclismo">
</datalist>
```

También es posible visualizar una etiqueta asociada a cada valor:

```
<input type="text" list="tratamiento">
<datalist id=" tratamiento ">
    <option label="Sr" value="Señor">
    <option label="Sra" value="Señora">
</datalist>
```

Las opciones pueden guardarse en un archivo xml y enlazar el elemento `<datalist>` a dicho archivo[18].

multiple:

Para los campos de tipo archivo, será posible seleccionar más de un archivo de una vez:

```
<input type="file" multiple="multiple">
```

Para ofrecer un método alternativo a navegadores que no soporten este atributo, se puede utilizar un script js, como el jQuery Multiple File Upload Plugin [19]

output

El campo `output` se utiliza para representar el resultado de una operación (por ejemplo, una multiplicación de 2 campos, cálculo de edad[20], etc... mediante javascript)

17 http://dev.opera.com/articles/view/an-html5-style-google-suggest/
18 http://html5.org/demos/dev.opera.com/article-examples.html
19 http://www.fyneworks.com/jquery/multiple-file-upload/
20 http://www.pageresource.com/html5/output-tutorial/

keygen

Crea una pareja de Clave Pública y Clave Privada[21].

La clave privada se encripta y se almacena en la base de datos local de contraseñas. La clave pública se envía con el formulario.

progress y meter

Estos dos elementos son similares y se utilizan para representar un progreso.

Normalmente `progress` representa una barra de progresos, para indicar el porcentaje completado de una tarea, mientras que `meter` es un indicador genérico[22].

Nuevos Atributos

Min y Max

Determina el valor máximo y mínimo que puede tener un campo (tal y como hemos visto en los `input` de tipo `number` y `range`):

```
<input name="edad" type="number" min="18" max="25">
```

Step

Indica el incremento entre un valor y el siguiente:

```
<input name="tiempo-consulta" type="number" min="15" max="60"
step="15">
```

placeholder

Permite incluir un texto a modo de ayuda para rellenar un campo de formulario:

```
<input name="termino" placeholder="Buscar en la web">
```

La diferencia entre `placeholder` y el atributo `value` (que utilizábamos anteriormente), es que no existe valor por defecto en el campo y cuando el usuario gane el foco de este campo, el texto desaparece.

El atributo `placeholder` no debe usarse como alternativa al elemento `<label>` Cada uno tiene una misión diferente.

autofocus

Permite forzar al navegador a que sitúe el foco en un campo de formulario determinado, de manera nativa (sin necesidad de utilizar javascript):

```
<input name="termino" autofocus>
```

21 http://wufoo.com/html5/elements/4-keygen.html

22 http://html5doctor.com/measure-up-with-the-meter-tag/

autocomplete

Se utiliza para activar o desactivar el autocompletado de un formulario o de un campo concreto:

```
<form autocomplete="off">
<input type="text" name="nombre" autocomplete="off">
```

spellcheck[23]

Activa o desactiva la propiedad de revisión ortográfica en un contenido editable: `textarea o un input type="text"`:

```
<input type="text" spellcheck="false">
<textarea spellcheck="true">
```

Validación de formularios[24] [25]

Además de las validaciones que proporcionan por sí mismos los campos y atributos vistos hasta ahora, existen 3 atributos extra para la validación de un formulario:

required

Con el objetivo de facilitar la validación del formulario, el atributo `required` obliga a rellenar el campo al que se aplica el atributo.

```
<input type="text" name="usuario" required>
```

Este atributo sólo se puede aplicar a los campos de tipo `text, search, url, telephone, email, password, date pickers, number, checkbox, radio, y file`.

novalidate

Este atributo especifica que un formulario o un campo `input` no se debe validar cuando se ejecute el formulario.

```
<form action=" " novalidate="novalidate">
E-mail: <input type="email" name="user_email">
<input type="submit" value="enviar">
</form>
```

Este atributo sólo se puede aplicar a: `form, text, search, url, telephone, email, password, date pickers, range, y color`.

23 http://blog.whatwg.org/the-road-to-html-5-spellchecking
24 http://www.alistapart.com/d/forward-thinking-form-validation/enhanced.html
25 http://blog.mozilla.com/webdev/2011/03/14/html5-form-validation-on-sumo/

pattern

Podemos crear un patrón de entrada de datos, basado en expresiones regulares, para un campo determinado, con el fin de que sea el propio navegador quien realice la validación de ese campo, en base al patrón definido, sin necesidad de utilizar una validación javascript:

```
<input type="text" name="codigo-idioma" pattern="[A-z]{2}"
title="Código de idioma con 2 letras">
```

```
<input id="telefono" name="telefono" type="tel"
placeholder="Patron: 1-234-567-8910" required size="50"
pattern="([0-9]{1}(-[0-9]{3})(-[0-9]{3})(-[0-9]{4}))">
```

Este atributo sólo se puede aplicar a los campos de tipo `text, search, url, telephone, email y password`.

accept

El atributo `accept` se utiliza para limitar el formato de los archivos válidos en un `input type="file"`, en base a su MIME type (audio/*, video/*, image/*...):

```
<input type=file accept="image/*">
```

CSS aplicable a los nuevos campos/atributos

Podremos aplicar CSS a los campos de formulario en función del valor de sus atributos, por ejemplo:

```
input[type=submit] { ... }
input:required { ... }
input:disabled { ... }
input:checked + label { ... }
input:read-only { ... }
input[type=text]:focus:valid { ... }
input[type=email]:focus:invalid { ... }
input[type=number]:focus:in-range { ... }
input[type=number]:focus:out-of-range  { ... }
```

O ir más allá y hacer filtros[26]...

[26] http://tympanus.net/Tutorials/CSS3FilterFunctionality/

4. AUDIO Y VIDEO

Con HTML 5 es posible insertar audio[27] y video[28] de forma nativa [29], sin depender de plugins de ningún tipo (flash, silverlight…) [30]:

Antes:

```
<object classid="clsid:xxxxyyyyyzzzzzz" width="400" height="300"
codebase="http://download.macromedia.com/pub/shockwave/cabs/flash/swflas
h.cab#version=6,0,40,0">
<param name="allowFullScreen" value="true" />
<param name="allowscriptaccess" value="always" />
<param name="src" value="http://www.youtube.com/v/ACCDDCC" />
<param name="allowfullscreen" value="true" />
<embed type="application/x-shockwave-flash" width="400" height="300"
src="http://www.youtube.com/v/ ACCDDCC " allowscriptaccess="always"
allowfullscreen="true">
</embed>
</object>
```

Ahora:

```
<video width="640"  height="360"
src="http://www.youtube.com/v/ACCDDCC.mp4" controls autobuffer>
    <p>
      Alternativa para navegadores sin soporte:
      <a href=" http://www.youtube.com/v/ACCDDCC.mp4"> </a>
    </p>
</video>
<audio src="musica.ogg" controls autobuffer >
    <a href="musica.ogg">Descarga audio</a>
</audio>
```

Atributos:

En el reproductor de audio/video podemos utilizar los siguientes atributos:

src

La URL del contenido de audio/video (puede reemplazarse por `source`).

```
<audio src="cancion.mp3"></audio>
```

autoplay (true|false):

Indica si el archivo debe reproducirse automáticamente o no.

```
<audio src="cancion.mp3" autoplay="false"></audio>
```

loop (true|false):

Indica si el archivo debe volverse a reproducir una vez llegado a su fin.

27 http://html5doctor.com/native-audio-in-the-browser/
28 http://html5doctor.com/the-video-element/
29 http://dev.opera.com/articles/view/everything-you-need-to-know-about-html5-video-and-audio/
30 http://www.youtube.com/html5

```
<audio src="cancion.mp3" loop="true"></audio>
```

preload (none|metadata|auto):

Indica si el archivo debe pre-cargarse o no.

La diferencia entre `metadata` y `auto` es que `metadata` sólo pre-carga el archivo y con `auto`, será el navegador quien decida si pre-carga todo el archivo.

Antes de este atributo, se utilizaba el atributo `autobuffer`. Para mayor compatibilidad, es recomendable usar ambos atributos conjuntamente (`autobuffer` y `preload`).

```
<video src="video.mp4" preload="auto" autobuffer></video>
```

controls:

Indica si se deben mostrar los controles de reproducción o no.

```
<video src="video.mp4" controls></video>
```

source

Debido al problema de soporte de formatos entre los diferentes navegadores, es recomendable ofrecer el contenido en diferentes formatos. Para indicarle al navegador cuáles son, prescindiremos del atributo `src` (ya que sólo puede enlazar con un único archivo) y lo reemplazaremos por varias instancias del atributo `source` (tantas como formatos dispongamos):

```
<audio controls>
    <source src="cancion.mp3">
    <source src="cancion.ogg">
<!—solución si no hay soporte (enlace descarga, flash...) -->
</audio>
```

poster

Si el video no se carga por alguna razón, podemos establecer una imagen estática (un fotograma del video) que se muestre en ese caso.

```
<video width="250" height="200" src="video.mp4" controls
poster="imagen.png">
</video>
```

Controles personalizados:

Es posible programar y diseñar nuestros propios controles personalizados para el reproductor de audio/video.

Para ello, será necesario programarlo con javascript, utilizando los atributos y métodos habilitados para tal efecto[31] [32], o bien utilizar alguna librería ya creada[33].

Accesibilidad en los elementos Audio y Video:

Aunque estemos hablando de un soporte nativo de audio y video por parte de los navegadores, sin ningún plugin externo aparte, los problemas de accesibilidad ligados al propio contenido multimedia siguen estando presentes, además de los problemas derivados de los controles del reproductor.

Además de dar la correspondiente alternativa textual al contenido de audio/video, será necesario que el archivo no se reproduzca automáticamente y los controles puedan ser activados no sólo con el ratón, sino también con teclado o con cualquier otro dispositivo de entrada [34] [35].

Soporte de video:

Inicialmente[36], la idea era que todos los navegadores soportaran tanto Ogg Theora como H.264, pero:

Ogg es open source y libre de licencia.

H.264 está patentado por MPEG.

Google lanzó WebM[37] en 2010, un nuevo formato[38], con el que empezó otra guerra de formatos[39] [40].

Pero la guerra parece que llegaba a su fin. En marzo de 2012, Mozilla anunció[41] que finalmente iba a dar soporte al formato H.264 en futuras versiones de Firefox.

Soporte de audio:

En el caso de Audio, nos encontramos hasta 4 formatos diferentes, con un soporte muy pobre por parte de los navegadores.

---

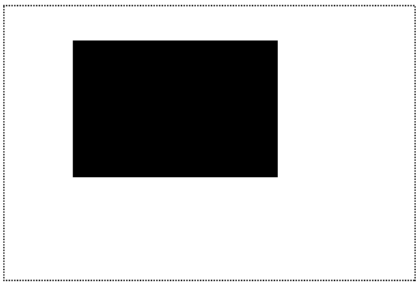
31 http://dev.opera.com/articles/view/everything-you-need-to-know-about-html5-video-and-audio/
32 http://www.broken-links.com/2009/10/06/building-html5-video-controls-with-javascript/
33 http://mooplay.challet.eu/
34 http://dev.opera.com/articles/view/accessible-html5-video-with-javascripted-captions/
35 http://www.brucelawson.co.uk/2009/accessibility-of-html5-video-and-audio-elements/
36 http://www.historiasdequeso.com/2009/06/sobre-youtube-html5-h264-y-theora.html
37 http://www.webmproject.org/
38 http://www.anieto2k.com/2010/05/19/webm-el-codec-de-video-open-source-de-google/
39 http://www.anexom.es/servicios-en-la-red/videos-online/la-guerra-del-video-en-html5-h-264-ogg-theora-y-webm-i/
40 http://www.anexom.es/servicios-en-la-red/videos-online/la-guerra-del-video-en-html5-h-264-ogg-theora-y-webm-y-ii/
41 http://blog.lizardwrangler.com/2012/03/18/video-user-experience-and-our-mission/

5. CANVAS

El elemento <canvas>[42] es un mapa de bits que puede usarse para renderizar gráficos, juegos, o cualquier otra imagen visual, en tiempo real.

```
<canvas id="canvas-prueba" width="300" height="200"></canvas>
```

Es un "lienzo en blanco" sobre el que podemos dibujar lo que queramos, e interactuar con él (mediante javascript).



Dibuja!!

```
<script type="text/javascript">
    function dibuja() {
       var b_canvas = document.getElementById("canvas-prueba");
       var b_context = b_canvas.getContext("2d");
       b_context.fillRect(50, 25, 150, 100);
    }
</script>
<p>
    <a href="#" onclick="dibuja();return false">Dibuja!</a>
</p>
```

API 2D

Relleno, borde y líneas básicas:

Relleno: `fillStyle`

Borde: `strokeStyle`

Grosor de las líneas/bordes: `lineWidth`

Rectángulos rellenos: `fillRect`

Rectángulos vacíos (sólo borde): `strokeRect`

Limpiar una parte de canvas: `clearRect`

[42] http://diveintohtml5.info/canvas.html

```
context.fillStyle   = '#abc';
context.strokeStyle = '#000';
context.lineWidth   = 2;

//x,y,width,height
context.fillRect  (0, 0,  70, 40);
context.strokeRect(50,10, 50, 35);
context.clearRect (30,25, 30, 10);
```

Paths (caminos):

Gracias a la propiedad Paths, podemos dibujar formas personalizadas.

Primero dibujaremos el contorno y estableceremos los atributos de relleno y borde.

Después, declararemos el comienzo del camino con `beginPath()` y procederemos a definirlo. Una vez finalizado, tendremos que aplicar el relleno y borde definido y cerrar el camino con `closePath()`.

```
context.fillStyle   = '#abc';
context.strokeStyle = '#000';
context.lineWidth   = 2;


context.beginPath();
// Coordenadas (x,y)
context.moveTo(10, 10);
context.lineTo(100, 10);
context.lineTo(10, 100);
context.lineTo(10, 10);

context.fill();
context.stroke();
context.closePath();
```

Inserción de imágenes:

Podemos insertar imágenes en el `canvas` mediante la propiedad `drawImage` y varios argumentos:

Imagen básica: fuente de la imagen y coordenadas X,Y para situarla en `canvas`.

Imagen media: los 3 argumentos de la imagen básica, más la anchura y altura de la imagen.

Imagen avanzada: los 5 argumentos anteriores y 4 más: coordenadas X,Y, altura y anchura dentro de la imagen. De este modo, podemos recortar dinámicamente la imagen y mostrar únicamente la porción que deseemos en el `canvas`.

```
// Básica: imagen, coord canvas. x , coord canvas. y).
context.drawImage(img_elem, dx, dy);

// Media: imagen, coord canvas. x , coord canvas. y, width,
height.
context.drawImage(img_elem, dx, dy, dw, dh);

// Avanzada: imagen, coord. imagen x , coord. imagen y, width
imagen, height imagen, coord canvas. x , coord canvas. y, width,
height.
context.drawImage(img_elem, sx, sy, sw, sh, dx, dy, dw, dh);
```

Otras posibilidades:

No hay límites para el elemento canvas, podemos insertar texto[43], pintar[44], manipular elementos...

Las posibilidades son infinitas!! [45] [46] [47] [48] [49] [50]

43 http://dev.opera.com/articles/view/html-5-canvas-the-basics/
44 http://dev.opera.com/articles/view/html5-canvas-painting/
45 http://www.phpguru.org/static/html5-canvas-examples
46 http://craftymind.com/factory/html5video/CanvasVideo.html
47 http://mugtug.com/sketchpad/
48 http://alteredqualia.com/canvasmol/
49 http://www.kesiev.com/akihabara/
50 http://9elements.com/io/projects/html5/canvas/

7. API's

Geolocalización

La geolocalización consiste en averiguar en qué lugar del mundo nos encontramos exactamente (mediante la dirección IP, conexión de red inalámbrica, torre de señal móvil, GPS, etc.).

HTML está desarrollando una API para que podamos detectar desde el navegador[51] la posición exacta.

Por ejemplo:

```
function get_location() {
    navigator.geolocation.getCurrentPosition(funcion);
}
```

Drag&Drop

Permite arrastrar y agarrar elementos en una página web.

En combinación con Javascript, podremos darle diferentes utilidades[52] [53], como por ejemplo, un carrito de la compra[54], una ordenación de contenido[55], etc[56].

Almacenamiento (local, sesión y base de datos)

Mediante los nuevos sistemas de almacenamiento de HTML 5, vamos a poder almacenar información en el navegador.

Las cookies, que actualmente utilizamos, están pensadas para almacenar una información escasa y los navegadores las envían al servidor cada vez que se recarga la página, por lo que se consume tiempo y ancho de banda extras.

---

51 http://diveintohtml5.info/detect.html#geolocation
52 http://decafbad.com/2009/07/drag-and-drop/api-demos.html
53 http://ljouanneau.com/lab/html5/demodragdrop.html
54 http://nettutsplus.s3.amazonaws.com/64_html5dragdrop/demo/index.html
55 http://decafbad.com/2009/07/drag-and-drop/outline.html
56 http://web.ontuts.com/tutoriales/drag-drop-en-html-5/

Por lo tanto, se necesita un nuevo sistema para poder almacenar gran cantidad de información y que el intercambio de esta información entre el navegador y el servidor sea rápida y eficaz.

Existen 3 tipos de almacenamiento[57]:

Local: para almacenar datos (sólo pares clave/valor) en la máquina del usuario. Los datos almacenados son únicos al dominio (preferencias).

Sesión: para almacenar datos (sólo pares clave/valor) únicamente válidos durante la sesión (carritos de la compra o estado de aplicación).

Base de datos: para almacenar datos relacionales ofreciendo una API de base de datos SQL, con todo lo que ello implica.

Con el tipo de almacenamiento adecuado, podremos desarrollar nuestras aplicaciones fácilmente.

Algunos casos de uso son presentaciones[58], juegos[59], etc..

57 http://theproc.es/2010/4/16/18070/almacenamiento-con-html5:-almacenamiento-local

58 http://slides.html5rocks.com

59 http://chrome.angrybirds.com/

Enpresa Digitala

PARTE 2: SOPORTE

1. SOPORTE EN NAVEGADORES

Los navegadores que aún no soportan HTML 5 van a necesitar una ayuda para que rendericen correctamente los elementos.

Detección de soporte HTML 5

* *Modernizr*[60]: librería JavaScript con licencia MIT que detecta la compatibilidad de nuestro navegador con HTML5 y CSS3

Detección de soporte para `autofocus`[61], por ejemplo.

También podemos conocer si nuestro navegador actual[62] soporta HTML 5

Declaración CSS:

```
/* Declarando elementos HTML 5 */
article,aside,canvas,details,figcaption,figure,footer,header,hgro
up,menu,nav,section,summary{  display: block;  }
```

Para IE:

* HTML5 Shiv [63] [64]

```
<!--[if lt IE 9]><script
src="//html5shiv.googlecode.com/svn/trunk/html5.js">
</script>
<![endif]-->
```

* *IE Print Protector*[65] (sólo para impresión)

* IE Canvas[66]

60 http://www.modernizr.com/
61 http://diveintohtml5.info/detect.html#input-autofocus
62 http://html5test.com/
63 http://code.google.com/p/html5shiv/
64 http://remysharp.com/2009/01/07/html5-enabling-script/
65 http://www.iecss.com/print-protector/
66 http://code.google.com/p/explorercanvas/

2. MÁS RECURSOS

Polyfills:

https://github.com/Modernizr/Modernizr/wiki/HTML5-Cross-browser-Polyfills

Tablas de soporte:

http://html5please.us/

http://caniuse.com/

http://www.fmbip.com/litmus

http://html5test.com/

http://miketaylr.com/code/input-type-attr.html

Enlaces indispensables:

http://dev.opera.com/articles/tags/html5/

https://developer.mozilla.org/en/HTML/HTML5

http://wufoo.com/html5/

http://html5demos.com/

http://www.pageresource.com/html5tutorials.html

http://diveintohtml5.info

http://html5doctor.com

http://www.w3conversions.com/resources.html (prácticamente están todos los recursos unificados)

Enpresa Digitala

CSS 3

PARTE 1: INTRODUCCIÓN CSS 3

1. INTRODUCCIÓN:

Tras 10 años sin evolución alguna en la especificación CSS[67], el W3C ha puesto en marcha la nueva especificación CSS3.

- Diciembre 1996: **CSS 1**
- Mayo 1998: **CSS 2**
- Septiembre 2009: **CSS 2.1**

Esta nueva especificación engloba la práctica totalidad de la especificación CSS 2, pero añade nuevas propiedades.

Estado actual de la especificación CSS 3:

Para evitar que la nueva especificación se demore en exceso, CSS 3 se ha dividido en diversos módulos[68] (p.e.):

CSS Values and Units	CSS Backgrounds and Borders
CSS Cascading and Inheritance	CSS Fonts
CSS Text	CSS Basic Box Model
CSS Text Layout	CSS Multi-column Layout
CSS Line Grid	CSS Template Layout

Cada módulo tiene sus propios editores y su propio ritmo, por lo que algunos módulos ya han pasado a ser candidatos a recomendación, y otros acaban de empezar a trabajar:

Stable	Current	Upcoming	Notes
Media Queries	CR	PR	
CSS Style Attributes	CR	PR	
Testing	**Current**	**Upcoming**	**Notes**
CSS Backgrounds and Borders Level 3	CR	CR	
CSS Marquee	CR	PR	
CSS Multi-column Layout	CR	CR	
CSS Mobile Profile 2.0	CR	PR	Status unknown
CSS TV Profile 1.0	CR	?	Status unknown
Refining	**Current**	**Upcoming**	**Notes**
CSS Speech	LC	CR	
CSS 2D Transformations	WD	WD	
CSS Transitions	WD	WD	
CSS Print Profile	LC	?	Status unknown

67 http://www.w3.org/Style/CSS/
68 http://www.w3.org/Style/CSS/current-work

Soporte actual:

Cada motor de renderizado está adoptando las nuevas propiedades a su ritmo. Antes de dar soporte completo, cada motor implementa una versión "beta" de dicha propiedad.

La implementación por CSS se realiza utilizando unos **prefijos**[69] específicos (en función de cada motor de renderizado) delante de la propiedad CSS 3 a utilizar:

prefix	organization
`-ms-`, `mso-`	Microsoft
`-moz-`	Mozilla
`-o-`, `-xv-`	Opera Software
`-khtml-`	KDE
`-webkit-`	Apple
`-rim-`	Research In Motion

Soporte actual en los navegadores:

- Web Design Checklist[70]
- Can I Use?[71]
- Readiness[72]

Generadores automáticos generales:

- CSS3 Generator[73]
- CSS3 Click Chart[74]
- CSS3 Maker[75]

Edición en tiempo real

- Dabblet[76] y CSS3 please[77]
- JSFiddle[78]

69 http://www.w3.org/TR/CSS21/syndata.html#vendor-keywords
70 http://www.findmebyip.com/litmus/
71 http://caniuse.com/
72 http://html5readiness.com/
73 http://www.css3generator.com/
74 http://www.impressivewebs.com/css3-click-chart/
75 http://www.css3maker.com/
76 http://dabblet.com/
77 http://css3please.com/
78 http://jsfiddle.net/

Enpresa Digitala

PARTE 2: NOVEDADES

1. BORDES REDONDEADOS

Podemos cambiar la apariencia de nuestros bordes, de cuadrados a redondeados.

Se puede configurar el radio de la curvatura con el mismo valor para todos los lados del borde, o bien configurar uno diferente para cada lado, de manera individual:

```
border-radius

border-top-left-radius
border-top-right-radius
border-bottom-right-radius
border-bottom-left-radius
```

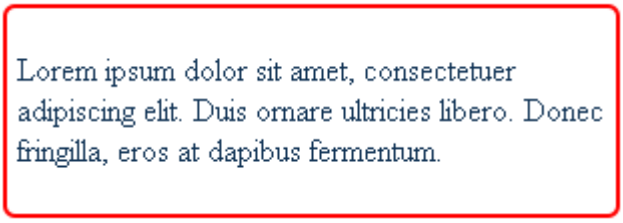
También podemos unificar las propiedades (en el sentido de las agujas de reloj):

```
border-radius: 10px 20px 30px 60px;
```

Propiedades temporales:

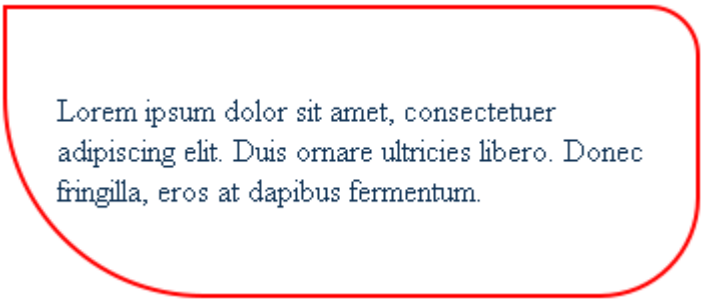
```
-webkit-border-radius

-webkit-border-top-left-radius
-webkit-border-top-right-radius
-webkit-border-bottom-right-radius
-webkit-border-bottom-left-radius

-moz-border-radius
-moz-border-radius-topleft
-moz-border-radius-topright
-moz-border-radius-bottomright
-moz-border-radius-bottomleft
```

Ejemplo 1:




```
#esquinas-redondeadas{
    border:2px solid #F00;

    border-radius:6px;
}
```

Ejemplo 2:



Forma unificada:

```
#esquinas-redondeadas-desiguales{
    border:2px solid #F00;

    border- radius: 0px 25px 50px 100px;
}
```

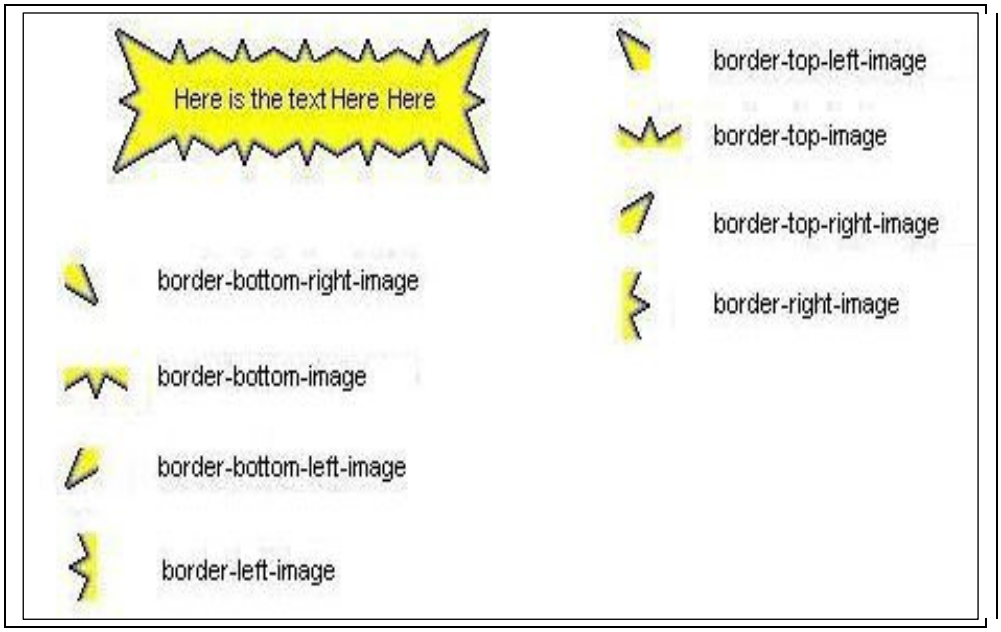
O la forma compleja:

```
#esquinas-redondeadas-desiguales{
    border:2px solid #F00;

    border-top-left-radius: 0px;
    border-top-right-radius: 25px;
    border-bottom-right-radius: 50px;
    border-bottom-left-radius: 100px;
}
```

2. IMÁGENES EN BORDES

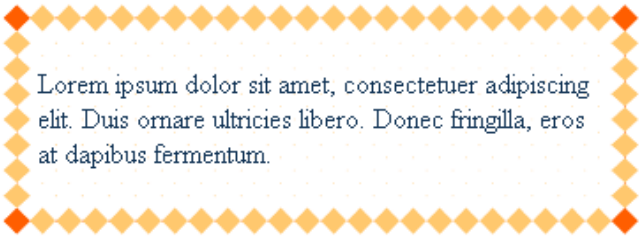
Podemos utilizar imágenes para crear el borde de un elemento

```
border-image
    border-top-image
    border-right-image
    border-bottom-image
    border-left-image


border-corner-image
    border-top-left-image
    border-top-right-image
    border-bottom-left-image
    border-bottom-right-image
```



Y decidir si queremos que se repita la imagen o bien se estire, mediante las propiedades `round` y `stretch`.

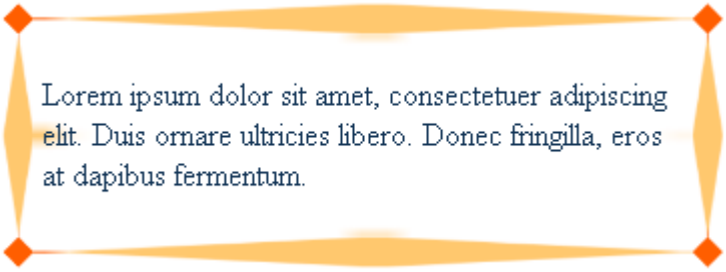
Ejemplo 1:




```
#borde-imagen{
    border:15px solid orange;

    border-image: url(border.png) 27 27 27 27 round round;
```

```
}
```

Ejemplo 2:




```
#borde-imagen2{
    border:15px solid orange;

    border-image: url(border.png) 27 27 27 27 stretch stretch;
}
```

También podemos utilizar generadores[79] automáticos[80].

79 http://border-image.com/
80 http://mentadreams.com/o/css3_border/

3. AJUSTES DE LOS BORDES

Podemos configurar el modo en que se repetirá la imagen de los bordes:

```
border-fit
    border-fit-length
    border-fit-width
```

`border-fit-length` determina la longitud de la imagen en los 4 lados:

```
border-fit-length
    border-top-fit-length
    border-right-fit-length
    border-bottom-fit-length
    border-left-fit-length
```

mientras que `border-fit-width` determina la anchura:

```
border-fit-width
    border-top-fit-width
    border-right-fit-width
    border-bottom-fit-width
    border-left-fit-width
```

Los valores de estas propiedades pueden ser: `clip`, `repeat`, `scale`, `stretch`, `overwrite`, `overflow`, `space`

También tenemos la propiedad `border-corner-fit` que determina la configuración de las imágenes en las esquinas.

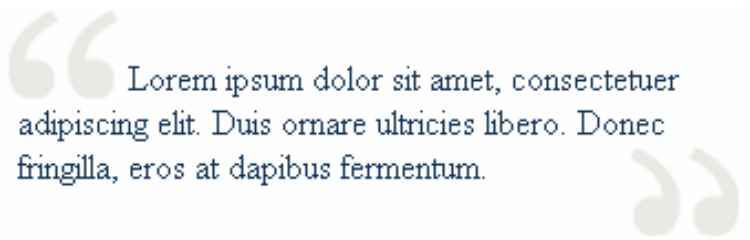
4. MÚLTIPLES IMÁGENES EN EL FONDO

Hasta ahora, sólo podíamos utilizar una única imagen como fondo de un elemento.

Pero con CSS 3, ya podemos utilizar varias imágenes como fondo del mismo elemento:

```
background:
    url(img) top left no-repeat,
    url(img) top right no-repeat,
    url(img) bottom left no-repeat,
    url(img) bottom right no-repeat;
```

Ejemplo:

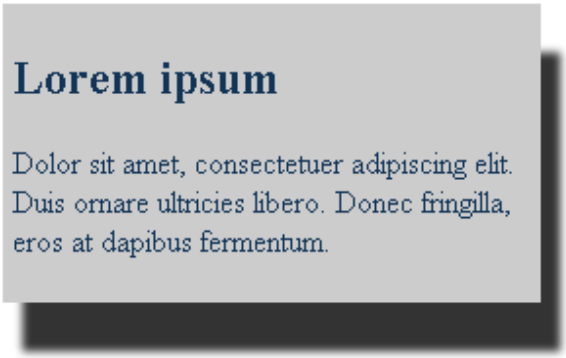



```
#multi-fondo{
    background: url(quote_open.gif) no-repeat top left,
url(quote_close.gif) no-repeat bottom right;
}
```

5. SOMBRAS EN BLOQUES

Para aplicar una sombra a un bloque, configuraremos el color, longitud horizontal y vertical, así como el radio de desenfoque de la sombra.

Ejemplo:



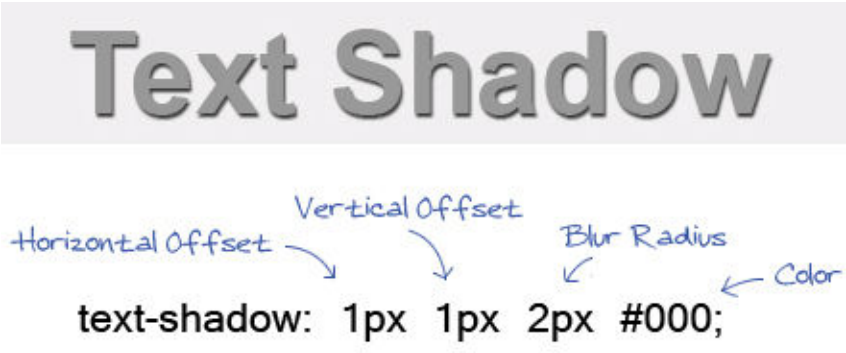

```
#bloque-sombra{
    box-shadow:10px 25px 5px #333;
}
```

Podemos usar generadores[81] automáticos[82]para que nos sea más cómoda la implementación.

81 http://www.layerstyles.org/builder.html
82 http://nicolasgallagher.com/css-drop-shadows-without-images/demo/

6. SOMBRAS EN TEXTOS

De la misma forma que podemos hacer sombras en los bloques, también vamos a poder aplicárselas a los textos:



También podremos aplicar varias sombras al mismo elemento

Ejemplo 1:

```
h2{
    text-shadow: 1px 5px 2px #F00;
}
```

Ejemplo 2:

```
h2{
    text-shadow: 0 1px 0 #fff, 0 -1px 0 #000;
}
```

También podemos usar un Generador Automático[83]

[83] http://westciv.com/tools/shadows/

7. CONTORNO DE TEXTO

Nota: a día de hoy, es una característica propietaria. No se recomienda su uso.

Podemos modificar un texto para que aparezca con un borde.

A este borde le podremos cambiar el color, grosor e incluso el relleno del texto (sobrescribiendo el color de texto que podría estar establecido previamente).

```
text-fill-color:
text-stroke:
    text-stroke-color:
    text-stroke-width:
```

Ejemplo:

Lorem Ipsum

```
#bloque-stroke h2 {
    text-fill-color: #ccc;
    text-stroke: #123456 2px; /* abreviado*/

    /* completo:*/
    /*
    text-stroke-color: #123456;
    text-stroke-width: 2px;
    */
}
```

También podemos usar un Generador Automático[84]

[84] http://westciv.com/tools/textStroke/index.html

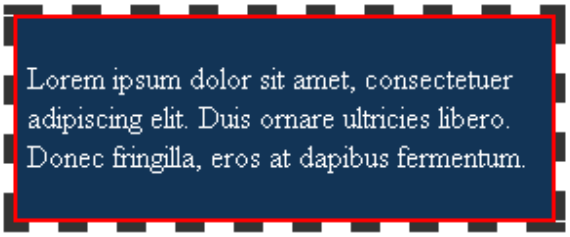
8. CONTORNO DE BLOQUES

El contorno de un elemento es similar al borde, con la diferencia de que no interfiere en el modelo de caja.

Por defecto, el contorno empieza justo por fuera del límite del borde, pero se puede ajustar a más distancia:

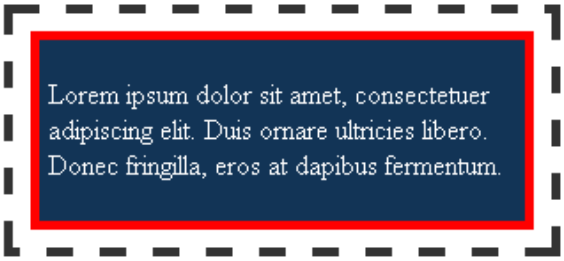
```
outline:
    outline-color:
    outline-style:
    outline-width:

outline-offset:
```

Ejemplo 1:




```
#outline{
    border:2px solid #F00;
    outline:5px dashed #333;
}
```

Ejemplo 2:




```
#outline2{
    border:2px solid #F00;
    outline:5px dashed #333;
    outline-offset: 10px;
}
```

9. TIPOGRAFÍAS PERSONALIZADAS

A partir de ahora vamos a poder utilizar la tipografía[85] que queramos en nuestras páginas (aunque el usuario no la tenga instalada en su dispositivo).

Para ello, primero tendremos que declarar la tipografía que vamos a utilizar, indicar dónde se encuentra y el formato de dicha fuente:

```
@font-face {
    font-family:'Adventpro';
    src: url(adventpro-Re.otf) format('opentype');
}
```

Y después, ya estamos listos para utilizarla en cualquier elemento (como si fuera una tipografía común):

```
h1{
    font-family: 'Adventpro', Helvetica, Sans-Serif;  }
```

Jugando con CSS 3

También podemos utilizar un generador automático[86], sobre todo para la compatibilidad entre diferentes navegadores[87].

Nota:

Cuidado con las licencias de las tipografías.

Antes de incluir una tipografía (que cualquier usuario va a poder descargar y usar posteriormente en su dispositivo), tendremos que asegurarnos de que su licencia nos permite incrustarla en una página web.

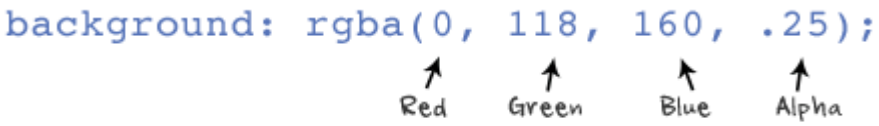
Repositorio de Fuentes de uso gratuito en web

Google Font Directory[88] y The League of Moveable Type[89].

85 http://perishablepress.com/press/2010/04/14/visual-walkthrough-font-face-css/
86 http://www.fontsquirrel.com/fontface/generator
87 http://webdesignerwall.com/general/font-face-solutions-suggestions
88 http://www.google.com/webfonts
89 http://www.theleagueofmoveabletype.com/

10. RGBA

Además de indicar el color en formato RGB (en vez de hexadecimal), también podremos modificar la opacidad del elemento:



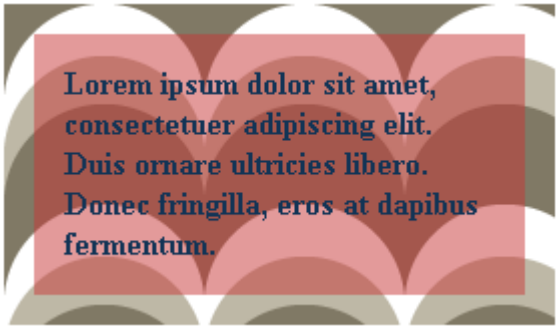
Los tres primeros valores indican la cantidad de Rojo, Verde y Azul de nuestro color.

El 4º valor, Alpha, indica el nivel de transparencia. Debe estar entre 0 y 1 (0 = transparente / 1 = opaco).

El color RGBA se puede utilizar en fondos, colores de primer plano, bordes, etc.

También podemos modificar la opacidad de un elemento, aunque definamos el color en hexadecimal. Bastaría con utilizar la propiedad `opacity`.

Ejemplo 1:




```
#bloque-transparencia-rgb{
    background:#FFF url(fondo2.gif);
}

#bloque-transparencia-rgb p{
    margin:10px;
    padding: 15px;
    font-weight:bold;
    background: rgba(200, 54, 54, 0.5);
}
```

Ejemplo 2:

```
#bloque-transparencia-rgb-bordes{
    background:#FFF url(fondo2.gif);
}

#bloque-transparencia-rgb-bordes p{
    margin:10px;
    padding: 15px;
    font-weight:bold;
    border:20px solid rgba(200, 54, 54, 0.5);
}
```

Ejemplo 3:

```
#bloque-transparencia{
    background:#FFF url(fondo.gif);
}

#bloque-transparencia p{
    color:#333;
    opacity: 0.8;
}
```

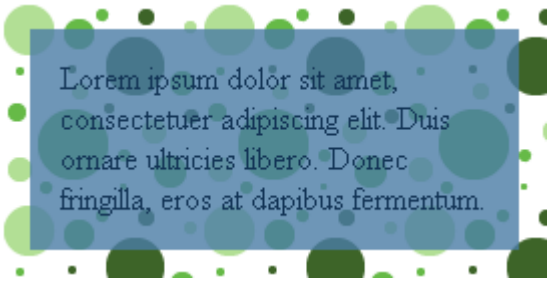
11. HSLA

Otra manera de indicar el color es mediante las propiedades de Tono (Hue), Saturación (Saturation) y Luminosidad (Lightness).

También se puede indicar la opacidad (Alpha), en el último valor:

```
background: hsla(H,S,L,A);
```

Ejemplo:




```
#bloque-hsla{
    background:#FFF url(fondo.gif);
}

#bloque-hsla p {
    margin:10px;
    padding: 15px;
    background: hsla(207,38%,47%,.8);
}
```

12. COLUMNAS MÚLTIPLES

A partir de ahora, podemos visualizar un texto en columnas[90].

Además del nº de columnas que queramos configurar, podemos indicar la separación entre ellas, dibujar una línea en esa separación y hasta el tamaño de cada columna:

```
column-count:
column-width:
column-gap:
column-rule:
```

Ejemplo:

Lorem ipsum dolor sit amet, augue sed dui vehicula posuere. Nullam sed mauris et sodales. Praesent vestibulum,

```
#bloque-columnas{
    column-count: 3;
    column-gap: 20px;
    column-rule: 1px solid #00000;
}
```

90 http://www.alistapart.com/articles/css3multicolumn

13. TAMAÑO DE BLOQUE

Podemos conseguir que un bloque no renderice siguiendo el modelo de caja. Por lo tanto, el tamaño del bloque permanecerá inalterable aunque implementemos bordes y/o márgenes.

```
box-sizing: border-box;
```

Para que vuelva a renderizar siguiendo el modelo de caja:

```
box-sizing: content-box;
```

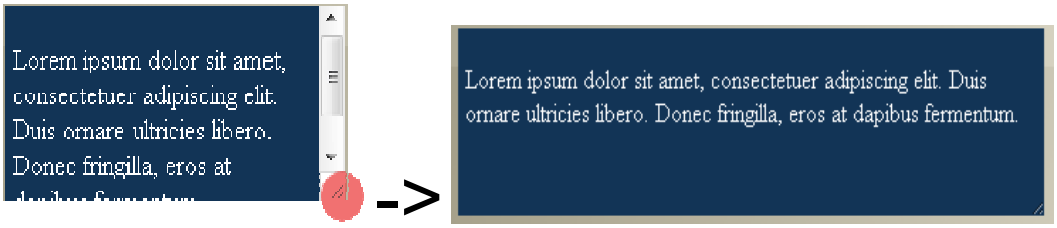
14. REDIMENSIONAMIENTO DE BLOQUE

Para aquellos bloques donde aparezca una barra de desplazamiento, podemos configurar la posibilidad de que el usuario redimensione el bloque según sus necesidades:

```
resize: both / horizontal / vertical;
```

Es aconsejable implementar una altura y anchura máximas, para que el redimensionamiento del usuario tenga unos límites.

Ejemplo:




```
#resize{
    width:200px; height:100px;
    overflow:auto;
    resize: both;
}
```

15. GRADIENTES

Podemos generar fondos con gradientes, sin necesidad de utilizar imágenes de fondo.

Podremos configurar múltiples[91] combinaciones[92] de colores, tamaños, direcciones...

-webkit-gradient(linear, 0% 0%, 0% 100%, from(#eee), to(#999));

-moz-linear-gradient(100% 100% 90deg, #999, #eee);

Webkit:

```
-webkit-gradient(<type>, <point> [, <radius>]?, <point> [,
<radius>]? [, <stop>]*)
```

Mozilla:

```
-moz-linear-gradient( [<point> || <angle>,]? <stop>, <stop> [,
<stop>]* )
```

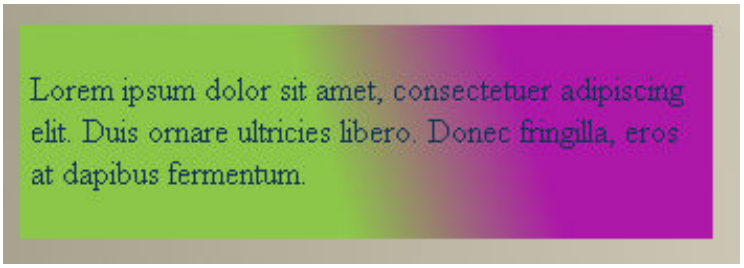
Ejemplo 1:

```
background: -webkit-gradient(linear, 0 0, 0 100%, from(red),
to(blue));
background: -moz-linear-gradient(top, red, blue);
```

91 http://www.the-art-of-web.com/css/linear-gradients/

92 http://www.the-art-of-web.com/css/radial-gradients/

Ejemplo 2:




```
#gradiente{
background: -webkit-gradient(linear,left bottom,right top,color-stop(0.44, rgb(140,199,73)),color-stop(0.72, rgb(173,24,168)));

background:-moz-linear-gradient(left bottom,rgb(140,199,73) 44%,rgb(173,24,168) 72%);
}
```

También podemos utilizar varios[93] generadores[94] automáticos[95], muy útiles para que sean compatibles con todos los navegadores que tienen soporte[96] y no sólo generar gradientes, al uso: podemos crear patrones[97] e incluso generar el patrón partiendo de una imagen concreta[98].

93 http://westciv.com/tools/gradients/
94 http://gradients.glrzad.com/
95 http://colorzilla.com/gradient-editor/
96 http://leaverou.me/demos/cssgradientsplease/
97 http://leaverou.me/css3patterns/
98 http://gradient-scanner.com/

16. REFLEJOS y MÁSCARAS

Nota: a día de hoy, son características propietarias. No se recomienda su uso.

REFLEJOS

Podemos conseguir que nuestro contenido se refleje, modificando diferentes parámetros: dirección, tamaño, opacidad, etc...

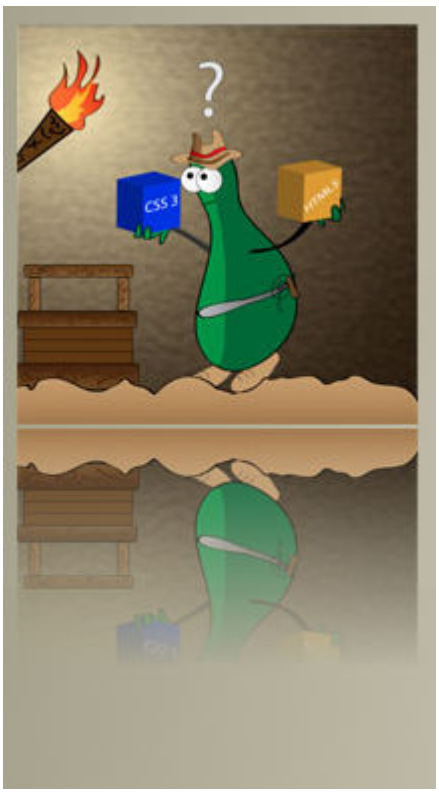
```
box-reflect: <direccion> <longitud>
```

`<direccion>`: puede llevar los siguientes valores: `above`, `below`, `left`, `right`.

`<longitud>`: longitud o porcentaje que especifica la distancia del reflejo desde el borde al bloque original. Si se omite, su valor es 0.

Combinaremos el reflejo con un gradiente para conseguir el efecto deseado[99].

Ejemplo 1:




```
#bloque-reflejo img{
    box-reflect: below 2px -webkit-gradient(linear, 0 0, 0 100%,
from(transparent),
color-stop(.4,transparent),to(white));
}
```

99 http://designshack.net/articles/css/mastering-css-reflections-in-webkit/

Ejemplo 2:

```
#bloque-reflejo-texto h2{
    box-reflect: below -5px -webkit-gradient(linear, left top,
left bottom, from(transparent),
to(rgba(255, 255, 255, 0.3)));
}
```

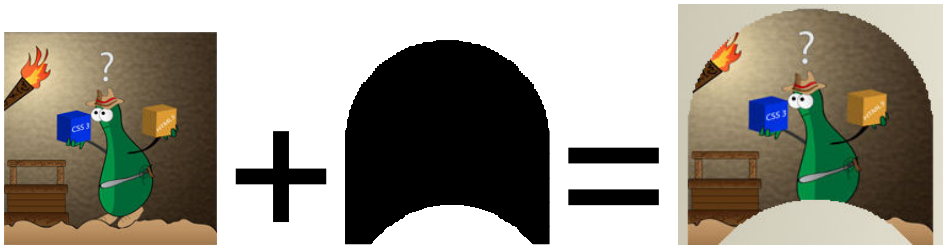
MÁSCARAS

Podemos utilizar una imagen para crear una máscara sobre un contenido, con diferentes parámetros:

```
mask (background)
mask-box-image (border-image)
mask-attachment (background-attachment)
mask-clip (background-clip)
mask-origin (background-origin)
mask-image (background-image)
mask-repeat (background-repeat)
mask-composite (background-composite)
```

También podemos combinar las máscaras con los gradientes[100].

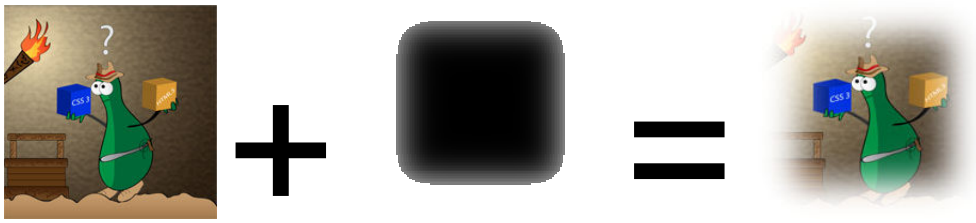
Ejemplo 1:



```
#bloque-mask img{
    mask-box-image: url(mascara.gif);
}
```

100 http://www.webkit.org/blog/181/css-masks/

Ejemplo 2:



```
#bloque-mask2 img{
    mask-box-image: url(black-mask.png) 75 stretch;
}
```

Ejemplo 3:

```
#bloque-mask3 img{
    border-radius: 10px;
    mask-image: -webkit-gradient(linear, left top, left bottom,
from(rgba(0,0,0,1)), to(rgba(0,0,0,0)));
}
```

17. FILTROS

Los filtros nos permiten manipular tanto elementos html como imágenes en varios aspectos: La sintaxis base es:

```
filter: filter(value);
```

Desenfoque: `blur(px)`

Brillo: `brightness(valor)` – de 0 a 1

Saturación: `saturate(%)`

Tono girado: `hue-rotate(deg)`

Contraste: `contrast(%)` -más de 100% añade contraste-

Invertir: `invert(%)`

Escala de grises: `grayscale(%)`

Sepia: `sepia(%)`

Opacidad: `opacity(%)`

Sombra: `drop-shadow(shadow)`

SVG: Usamos un elemento que contenga el estilo con el svg que queramos utilizar como filtro: `<filter id="contenedor-svg">...</filter>`

Después aplicamos ese elemento al elemento que queremos aplicar el filtro:

```
div {filter: url(#contenedor-svg); }
```

Además podremos combinar varios filtros a la vez:

```
div {filter: grayscale(100%) sepia(100%); }
```

Se puede utilizar un generado automático[101].

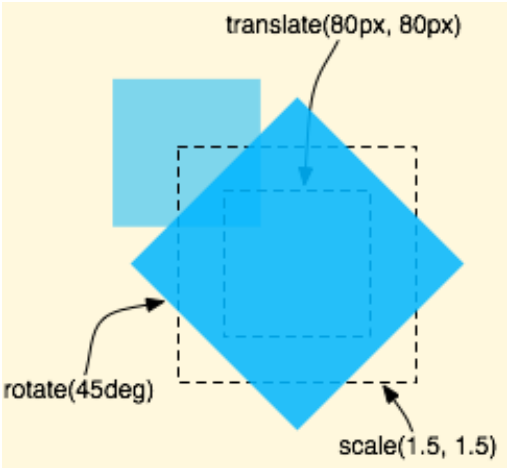
[101] http://html5-demos.appspot.com/static/css/filters/index.html

18. TRANSFORMACIONES 2D

Podemos usar diferentes transformaciones sobre un elemento (bloque y/o texto): rotaciones, escalado, desplazamiento y sesgado.

Lo mejor es que podemos combinarlas entre sí:

```
transform: rotate(deg);
transform: translate(x, y);
transform: scale(x, y);
transform: skew(deg, deg);
```



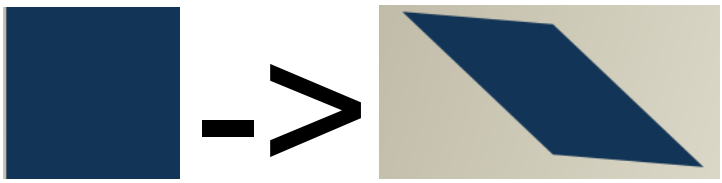
Ejemplo 1:

```
#menu li{
    transform: rotate(-5deg);
}
```

Ejemplo 2:

```
#menu li{
    transform: rotate(-5deg);
}

#menu li a{
    transform: rotate(5deg);
}
```

Ejemplo 3:




```
#transformacion2{
    transform: skew(45deg, 5deg);
}
```

MATRIX

Sirve para combinar varias transformaciones de una forma más eficaz, controlando los valores de forma matemática[102]:

```
#transformacion{
    transform: matrix(4.922, -0.944, 0.589, 3.944, 44.889,
70.000);
}
```

Podemos utilizar generadores automáticos[103] [104]

Iconos, formas

Estas transformaciones nos permiten simular[105] imágenes[106] sencillas[107] [108], simplemente con código css[109].

---

102 http://www.useragentman.com/blog/2011/01/07/css3-matrix-transform-for-the-mathematically-challenged/
103 http://www.useragentman.com/matrix/
104 http://meyerweb.com/eric/tools/matrix/

19. TRANSFORMACIONES 3D

La mayor parte de las transformaciones 2D tiene su versión 3D.

```
translate3d(x, y, z)
translateZ(z)

scale3d(sx, sy, sz)
scaleZ(sz)

rotateX(value)
rotateY(value)
rotate3d(x, y, z)

matrix3d(…)

perspective(value)
```

Para el caso de una matriz, los valores aumentan por lo que lo lógico es utilizar un generador automático[110].

Algunos ejemplos aclaratorios: [111] [112] [113] [114]

105 http://css3shapes.com/
106 http://pictos.drewwilson.com/
107 http://nicolasgallagher.com/pure-css-gui-icons/demo/
108 http://rathersplendid.net/home/pure-css-icons
109 http://www.imgtocss.com/
110 http://cssglue.com/matrix
111 https://www.webkit.org/blog/386/3d-transforms/
112 http://desandro.github.io/3dtransforms/
113 http://css-tricks.com/almanac/properties/p/perspective/
114 http://css-tricks.com/almanac/properties/p/perspective-origin/

20. TRANSICIONES

Las transiciones nos permiten cambiar propiedades CSS de un elemento durante una duración determinada: cambio de color, de posición, de opacidad… cualquier cosa que se nos ocurra. También se pueden combinar entre sí.

Las transiciones necesitan 4 atributos: propiedad, duración y tipo de transición y demora.

Propiedad:

Se puede aplicar la transición a una única propiedad (p.e. background) o bien a todas las propiedades de un elemento (all).

```
transition-property: nom_propiedad;
```

Duración:

Tenemos que indicar cuantos segundos dura la transición.

```
transition-duration: duración;
```

Demora (opcional):

Si queremos que la transición no empiece inmediatamente, podemos implementar una demora.

```
transition-delay: demora;
```

Tipos de Transiciones:

Siguen el patrón de las Curvas de Bezier[115]

Valores posibles[116]: default, linear, ease-in, ease-out, ease-in-out, cubic-bezier (estableciendo nosotros los valores).

```
transition-timing-function: tipo_transición;
```

Forma abreviada:

Es más útil indicar todas las propiedades de transiciones en una única declaración:

```
transition: nom_propiedad duración tipo_transición retraso;
```

Ejemplo 1: transición de color:

Inicialmente, nuestro bloque tiene un color de fondo. Al realizar un :hover sobre el bloque, éste cambiará de color.

Pero este cambio no se realizará instantáneamente, sino que tardará 2 segundos.

115 http://www.netzgesta.de/dev/cubic-bezier-timing-function.html
116 http://www.the-art-of-web.com/css/timing-function/

```
#bloque-transicion-1{
    background:#CCC;
    -webkit-transition: background 2s linear;
}

#bloque-transicion-1:hover{
    background:#333;
}
```

Ejemplo 2: transición de movimiento:

Inicialmente, nuestro bloque está en una determinada posición. Al realizar un :hover sobre el bloque, éste cambiará de posición. El cambio se realizará en 2 segundos.

```
#bloque-transicion-2{
    -webkit-transition: left 2s linear;
}

#bloque-transicion-2:hover{
    left:200px;
}
```

También podemos utilizar generadores[117] automáticos[118].

117 http://matthewlein.com/ceaser/

118 http://westciv.com/tools/transforms/index.html

21. KEYFRAMES

Las transformaciones/animaciones básicas que hemos visto, sólo permiten animar un elemento con un determinado tipo de transición.

Podemos añadir más complejidad, juntando varias transiciones en un mismo elemento[119], incluso interactuando varios elementos entre sí[120]. Para ello crearemos nuestra propia animación[121], a la cual invocaremos desde un elemento.

Declaración e invocación:

```
@keyframes MiAnimacion {
    0%   { opacity: 0; }
    100% { opacity: 1; }
}
#bloque {
    animation: MiAnimacion 5s infinite;
}
```

Si queremos que la animación tenga las mismas propiedades al inicio y al final, podemos agrupar los % en una misma declaración::

```
@keyframes MiAnimacion {
    0%, 100% {
       font-size: 10px;
    }
    50% {
       font-size: 12px;
    }
}
```

Propiedades

```
animation-name: MiAnimacion;
animation-duration: 4s;
animation-iteration-count: 10;
animation-direction: alternate;
animation-timing-function: ease-out;
animation-fill-mode: forwards;
animation-delay: 2s;
```

timing-function	ease, ease-out, ease-in, ease-in-out, linear, cubic-bezier(x1, y1, x2, y2) (e.g. cubic-bezier(0.5, 0.2, 0.3, 1.0))
duration & delay	Xs / Xms
duration-count	X
fill-mode	forwards, backwards, both, none
animation-direction	normal, alternate

119 http://coding.smashingmagazine.com/2011/05/17/an-introduction-to-css3-keyframe-animations/
120 http://www.standardista.com/forms/animation.html#slide1
121 https://developer.mozilla.org/en/css/css_animations

Podemos también escribir las propiedades resumidas. El orden no importa a excepción de la duración y el retraso. El primer tiempo será siempre el de la duración y el segundo, el del retraso:

```
animation: MiAnimacion 5s 1s 2 alternate backwards
```

Es posible combinar transformaciones con animaciones:

```
@keyframes MiAnimacion {
  from {
    -webkit-transform: rotate(0deg);
  }
  to {
    -webkit-transform: rotate(360deg);
  }
}
```

Uno de los usos principales de las animaciones sencillas va a ser la creación de banners[122] animados[123].

Animaciones Múltiples

Podemos crear animaciones múltiples, si separamos las invocaciones por comas:

```
.bloque {
   animation:
      MiAnimacion 2s infinite,
      MiOtraAnimacion 1s;
}
```

122 http://tympanus.net/Tutorials/AnimatedWebBanners/index.html

123 http://www.red-team-design.com/wp-content/uploads/2011/12/css3-animated-header-demo.html

22. MEDIA QUERY

Debido a la multitud de dispositivos desde los que accedemos a internet, las posibles dimensiones de la pantalla donde el usuario visualiza una web se han disparado. De ahí que haya nacido el concepto de Responsive Web Design.

Para que la una web se adapte a cada uno de los dispositivos, CSS3 permite comprobar las dimensiones de pantalla y cargar una css específica en función del tamaño[124].

También podremos comprobar si el dispositivo está en horizontal o en vertical, luminosidad, densidad de color, etc.

Ejemplo:

```
@media screen and (max-width: 980px) {

     #paginawrap {
        width: 95%;
     }

     #contenido {
        width: 60%;
        padding: 3% 4%;
     }

     #sidebar {
        width: 30%;
     }
}
```

Podemos ver varios[125] ejemplos[126] así como una galería[127] de páginas web que utilizan media querys.

124 http://webdesignerwall.com/tutorials/adaptive-responsive-design-with-css3-media-queries
125 http://stunningcss3.com/code/bakery/
126 http://w3conversions.com/sandbox/css3/transition/multibackground-bug.html
127 http://mediaqueri.es/

Enpresa Digitala

PARTE 3: SELECTORES

1. LISTA DE SELECTORES

* E – Elemento

* att – Atributo

* val – Valor

* n – Cualquier número entero.

Sintaxis	Descripción
`E [att^='val']`	Selecciona todos los elementos que inicien con un valor especificado. **Ejemplo:** a[href='http://miweb.com'] Seleccionará todos los enlaces hacia miweb.com
`E[att$='val']`	Selecciona todos los elementos que terminen con un valor especificado. **Ejemplo:** a[href$='.rar'] Seleccionará todos los enlaces de archivos rar.
`E[att*='val']`	Selecciona todos los elementos que contenga, indiferente a su ubicación, un valor especificado. **Ejemplo:** a[href*='tag'] Seleccionará todos los enlaces que contengan la palabra "tag".
`E:root`	Selecciona el elemento raíz de un documento. En HTML, el elemento raíz es siempre <html>
`E:nth-child(n)`	Selecciona el enésimo hijo de su elemento padre. **Ejemplo:** p a:nth-child(2) seleccionará el segundo enlace de un párrafo.
`E:nth-last-child(n)`	Selecciona el enésimo hijo de su elemento padre empezando a constar del último al primero. **Ejemplo:** p a:nth-last-child(2) seleccionará el penúltimo enlace de un párrafo.
`E:nth-of-type(n)`	Selecciona el enésimo elemento de su tipo. **Ejemplo:** p img:nth-of-type(1) seleccionará la primer imagen en un párrafo.
`E:nth-last-of-type(n)`	Selecciona el enésimo elemento de su tipo, empezando a constar del último al primero. **Ejemplo:** p img:nth-last-of-type(1) seleccionará la última imagen en un párrafo.
`E:last-child`	Selecciona el último hijo de un elemento. **Ejemplo:** .post p:last-child seleccionará el último párrafo dentro de un elemento con clase "post".
`E:first-of-type`	Selecciona el primer elemento de su tipo en el elemento padre. **Ejemplo:** .post img:first-of-type seleccionará la primer imagen dentro en un elemento con clase "post".
`E:last-of-type`	Selecciona el último elemento de su tipo en el elemento padre.

	Ejemplo: .post img:last-of-type seleccionará la última imagen dentro en un elemento con clase "post".
`E:only-child`	Selecciona el único elemento hijo de un elemento padre. **Ejemplo:** ul li:only-child seleccionará el elemento una de lista, cuando la lista solo contenga un elemento.
`E:only-of-type`	Selecciona el único elemento de cierto tipo. **Ejemplo:** .post img:only-of-type seleccionará las imágenes presentes en los elementos .post con solo una imagen.
`E:empty`	Selecciona los elementos web que no tienen elementos hijos. **Ejemplo:** ul:empty seleccionará todas las listas no numeradas sin elementos.
`E:target`	Selecciona los elementos que tienen como destino una URL .
`E:enabled`	Selecciona elementos (de formularios) con valor enabled (habilitado). **Ejemplo:** input[type="text"]:enabled seleccionará los campos de tipo texto que están habilitados.
`E:disabled`	Selecciona elementos (de formularios) con valor disabled (deshabilitado). **Ejemplo:** input[type="text"]:disabled seleccionará los campos de tipo texto que están deshabilitados.
`E:checked`	Selecciona elementos (de formularios) con valor checked (seleccionado). **Ejemplo:** input:checked seleccionará los campos que estén seleccionados.
`E::selection`	Selecciona los elementos que han sido seleccionados/resaltados por el usuario. Las propiedades aplicables son color, background, cursor y outline. **Ejemplo:** ::selection ubicara el texto que haya sido seleccionado/resaltado (con el cursor del mouse) por el autor.
`E:not(s)`	Selecciona todo los elementos que no sean un selector indicado (s) dentro de un elemento web. **Ejemplo:** .post:not(img) seleccionará todos los elementos que no sean imágenes y estén dentro del elemento web con clase "post".
`E ~ F`	Selecciona cualquier elemento F que esté precedido por el elemento F

CSS 3 Specifity[128]

[128] http://www.standardista.com/css3/css-specificity/

Enpresa Digitala

Parte 4: SOPORTE Y RECURSOS

1. SOPORTE

Prefijos de vendedores por Javascript:

CSS prefixer[129]

Mejorar el soporte en navegadores:

- CSS3 Pie[130]
- CSS SandPaper[131]
- Selectivizr[132]
- JQuery Extended Selectors[133]
- JQuery Animate Enhaced[134]
- CSS3 MediaQueries[135]
- Normalize[136]
- HTML5 Boilerplate[137]

Detectar si existe o no soporte

- Modernizr[138]
- Yepnope[139]

129 http://cssprefixer.appspot.com/
130 http://css3pie.com/
131 http://www.useragentman.com/blog/csssandpaper-a-css3-javascript-library/
132 http://selectivizr.com/
133 https://github.com/keithclark/JQuery-Extended-Selectors
134 http://playground.benbarnett.net/jquery-animate-enhanced/
135 http://code.google.com/p/css3-mediaqueries-js/
136 http://necolas.github.com/normalize.css/
137 http://html5boilerplate.com/
138 http://www.modernizr.com/
139 http://yepnopejs.com/

2. RECURSOS ÚTILES

Herramientas útiles:

- Resize my browser: redimensionar el navegador[140]
- Responsive design Test: visualización de una web en diferentes tamaños[141]
- iOS Media Query Previewer: visualización de una web en iOS[142]
- Support Details: conocer al detalle las características de Sistema Operativo, Navegador, etc... de un equpo.[143]
- Tamaño de las barras del navegador[144]

Varios

- Text-align:centaur;[145]
- ¿Se tiene que ver una web exactamente igual en cada navegador? [146]
- ¿La experiencia de uso debe ser exactamente igual en cada navegador?[147]

Ejemplos

- Planetario[148]
- Sistema Solar[149]
- Teclado[150]
- Con Canvas parte 1[151] y parte 2[152]
- Gráfica 3D[153]
- Vinilos[154]
- Spotlight[155]
- Selección (NSFW)[156]

Ejemplos (animaciones)

- Gravedad[157]
- Meninas 3D[158]
- Coke[159]
- Futurama[160]

---

140 http://resizemybrowser.com/
141 http://mattkersley.com/responsive/
142 http://markboultondesign.com/tools/index.html
143 http://www.supportdetails.com/
144 http://howtallaremytoolbars.com/
145 http://textaligncentaur.com/
146 http://dowebsitesneedtolookexactlythesameineverybrowser.com/
147 http://dowebsitesneedtobeexperiencedexactlythesameineverybrowser.com/
148 http://mozillademos.org/demos/planetarium/demo.html
149 http://neography.com/journal/our-solar-system-in-css3/
150 http://dl.dropbox.com/u/921159/Keyboard/page.html
151 http://www.effectgames.com/demos/canvascycle/
152 http://www.ernestdelgado.com/public-tests/canvasphoto/demo/canvas.html
153 http://www.marcofolio.net/css/wicked_css3_3d_bar_chart.html
154 http://www.zurb.com/playground/sliding-vinyl
155 http://svay.com/experiences/css3-spotlight/
156 http://fichtre.net/yop.html
157 http://mrdoob.com/projects/chromeexperiments/google_gravity/
158 http://www.romancortes.com/blog/css-3d-meninas/
159 http://www.romancortes.com/blog/pure-css-coke-can/
160 http://www.cssplay.co.uk/menu/css3-animation.html

- Super Mario[161]
- Star Wars intro[162]
- Star Wars AT-AT[163]
- Spiderman[164]
- The man from Hollywood[165]

Ejemplos (juegos)

- Pacman [166]
- Mario Bros[167]
- Memory[168]
- Ajedrez[169]

Ejemplos (tipografías)

- Back to the future[170]
- Posters parte 1[171] y parte 2[172]

Ejemplos (imágenes)

- Imágenes parte 1[173] y parte 2[174]
- Rueda de color[175]
- Taza de café[176]

Y más ejemplos (recopilación)

- Demos parte 1[177], parte 2[178] y parte 3[179]

Último apunte: sobre las características propietarias[180]

161 http://cordobo.com/1662-pure-css-animated-3d-super-mario-icon/
162 http://www.gesteves.com/experiments/starwars.html
163 http://anthonycalzadilla.com/css3-ATAT/index.html
164 http://www.optimum7.com/css3-man/
165 http://lab.tylergaw.com/themanfromhollywood/
166 http://worldsbiggestpacman.com/
167 http://arcade.rawrbitrary.com/mario/
168 http://media.miekd.com/css3memory/
169 http://designindevelopment.com/css/css3-chess-board/
170 http://code.garron.us/css/BTTF_logo/
171 http://graphicpush.com/css3-poster-with-no-images
172 http://neography.com/experiment/type1/
173 http://coding.smashingmagazine.com/2010/07/12/css3-design-contest-results/
174 http://www.queness.com/post/4023/18-brilliant-pure-css-drawings
175 http://crysodenkirk.com/blog/2010/07/color-wheels-with-only-css3-and-primary-colors/
176 http://gabri.me/htmlcss/2010/css3-gradients-coffee-cup/
177 http://speckyboy.com/2010/05/21/10-mind-blowing-experimental-css3-techniques-and-demos/
178 http://webdesignerwall.com/trends/47-amazing-css3-animation-demos
179 http://www.socialblogr.com/2010/03/80-html5javascriptcss3svg-experiments.html
180 http://www.alistapart.com/articles/every-time-you-call-a-proprietary-feature-css3-a-kitten-dies/